Seguridad en Sistemas, Aplicaciones y el Big Data

Tema 4. Autenticación en las aplicaciones web

Índice

Esquema

Ideas clave

4.1. Introducción y objetivos

4.2. Autenticación HTTP

4.3. Autenticación integrada en dominio

4.4. Autenticación web

4.5. Autenticación single sing on

4.6. Autenticación. Lista de comprobación

4.7. Referencias bibliográficas

A fondo

Recomendaciones de seguridad para la autenticación

Protocolo de autenticación Kerberos

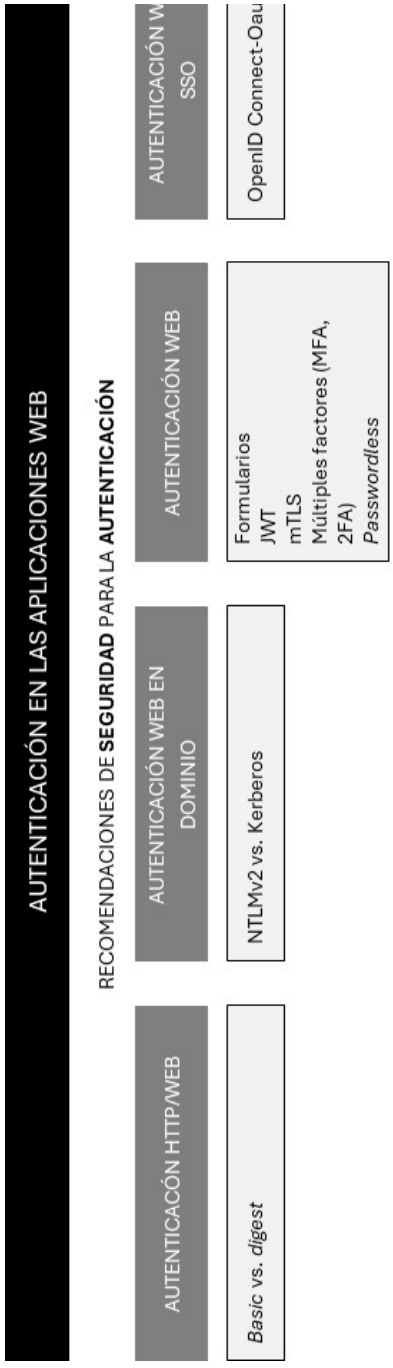
Autenticación y autorización en Azure App Service y Azure Functions

OWASP Transport Layer Security Cheat Sheet

Método de autenticación OpenID Connect

Test

Esquema



© Universidad Internacional de La Rioja (UNIR)

4.1. Introducción y objetivos

La autenticación es el proceso de verificar la identidad de un usuario. En las aplicaciones web, esto se logra, comúnmente, mediante la combinación de un identificador (como un nombre de usuario o correo electrónico) y una credencial (como una contraseña o un *token*). El objetivo es garantizar que solo los usuarios autorizados tengan acceso a los recursos de la aplicación. Los principales métodos de autenticación usados en las aplicaciones web son (Mcdonald, 2024):

- **Autenticación HTTP *basic:*** la autenticación HTTP *basic* es un método simple para proteger recursos web. El navegador solicita las credenciales al usuario (generalmente, nombre de usuario y contraseña) y las codifica en Base64. Envía esta cadena codificada en el encabezado Authorization de cada petición HTTP. El servidor decodifica las credenciales, las verifica y, si son correctas, permite el acceso.

- **Autenticación HTTP *digest:*** La autenticación HTTP *digest* es una mejora de la autenticación HTTP *basic.* A diferencia de *basic,* que envía las credenciales en texto plano, *digest* genera un *hash* de la contraseña, el nombre de usuario, el *nonce* (un valor aleatorio generado por el servidor) y otros datos. Este *hash,* en lugar de la contraseña en sí, se envía al servidor. El servidor realiza el mismo cálculo con la contraseña almacenada y el nonce para verificar que los hashes coincidan, lo que protege la contraseña de ser interceptada en la red.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Autenticación NTLMv2:** el método de autenticación NTLM (NT LAN Manager) es un protocolo de desafío respuesta desarrollado por Microsoft para la autenticación de usuarios en redes de Windows. En lugar de enviar la contraseña del usuario, el cliente genera un *hash* de la contraseña y lo utiliza para responder a un desafío (un valor aleatorio) enviado por el servidor. El servidor, que también tiene el *hash* de la contraseña del usuario, realiza el mismo cálculo para verificar la autenticidad del cliente. NTLM es un protocolo obsoleto que ha sido ampliamente reemplazado por Kerberos debido a sus vulnerabilidades de seguridad.

- **Autenticación Kerberos**: el protocolo Kerberos, de tipo SSO, es un sistema de autenticación de red que utiliza criptografía de clave secreta para asegurar la comunicación entre un cliente y un servidor a través de una red no segura. Se basa en un tercero de confianza (KDC) que emite tiques a los clientes para acceder a los servicios. En lugar de enviar la contraseña del cliente en la red, Kerberos la usa para obtener un tique del KDC que, a su vez, se usa para autenticar al cliente ante el servidor y , de esta forma, evitar que las credenciales reales se transmitan.

- **Autenticación basada en *token* (JWT):** la autenticación basada en *token* es un enfoque moderno que elimina la necesidad de sesiones en el servidor, haciéndolo ideal para arquitecturas de microservicios y API. El método más popular es JSON Web Token (JWT).

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **OAuth 2.0 y OpenID Connect (OIDC):** OAuth 2.0 es un marco de autenticación-autorización de tipo Single Sign On (SSO) que permite a una aplicación (el cliente) obtener acceso limitado a los recursos de un usuario en otro servicio (el servidor de recursos) sin compartir las credenciales del usuario, por ejemplo, cuando una aplicación pide permiso para acceder a tu perfil de Google. OpenID Connect (OIDC) es una capa de autenticación construida sobre OAuth 2.0. Proporciona una forma estándar para que las aplicaciones verifiquen la identidad de los usuarios y obtengan información básica del perfil. Al final del flujo, OIDC proporciona un ID *token* (un JWT) que contiene información del usuario, lo que permite a la aplicación–cliente autenticar al usuario.

- **Autenticación mutua basada en TLS:** la autenticación mutua basada en TLS *(transport layer security),* también conocida como mTLS, es un método en el que tanto el cliente como el servidor se autentican entre sí. Funciona mediante el intercambio y la validación de certificados digitales durante el *handshake* TLS. El cliente presenta un certificado al servidor y el servidor hace lo mismo. Ambos verifican que los certificados sean válidos y de confianza, garantizando una conexión cifrada y mutuamente autenticada. Es un método de alta seguridad utilizado comúnmente en microservicios y API.

- **Autenticación basada en formularios:** la autenticación basada en formularios es uno de los métodos más tradicionales. Se basa en la validación de las credenciales del usuario por la aplicación y, si son correctas, crea una sesión única para ese usuario que se almacena en una *cookie* del navegador del usuario y en la base de datos del servidor.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Autenticación multifactor (MFA):** la autenticación multifactor (MFA) o autenticación de dos factores (2FA) es una capa adicional de seguridad. Requiere que el usuario proporcione dos o más factores de verificación para acceder a una cuenta, conocimiento, posesión o inherencia.

- **Autenticación sin contraseña *(passwordless):*** la autenticación sin contraseña busca eliminar la dependencia de las contraseñas, que son un punto débil común. Enlaces mágicos, códigos únicos de un solo uso, son algunos ejemplos.

Los principales objetivos que se pretenden alcanzar en este tema son:

- Conocer los principales métodos de autenticación en aplicaciones web, sus vulnerabilidades y fortalezas.

- Conocer los principales tipos de ataque a los métodos de autenticación de las aplicaciones web.

- Analizar los mecanismos de defensa específicos de cada método de autenticación.

- Conocer los principales mecanismos de defensa generales para aplicar a los métodos de autenticación de las aplicaciones web.

© Universidad Internacional de La Rioja (UNIR)

4.2. Autenticación HTTP

Autenticación HTTP basic

La autenticación HTTP *basic* es el método más simple para **proteger** un recurso en un **servidor web.** Es una forma de autenticación de desafío–respuesta que se basa en la codificación Base64 de las credenciales de un usuario. Es importante destacar que no es un método seguro para su uso en entornos de producción, ya que las credenciales no se cifran, solo se **codifican.**

El **proceso** se desarrolla en los siguientes pasos:

- **Solicitud del cliente (sin credenciales):** el cliente (generalmente, un navegador web) realiza una solicitud GET o POST a un recurso protegido en el servidor.
- **Respuesta del servidor (desafío):** el servidor recibe la solicitud, detecta que el recurso requiere autenticación y responde con un código de estado 401 Unauthorized. La respuesta incluye un encabezado WWW-Authenticate: Basic realm="Nombre del Recurso". El *realm* es una cadena que el servidor usa para identificar el área protegida y que el navegador muestra al usuario en el cuadro de diálogo de inicio de sesión.
- **Introducción de credenciales por el usuario:** el navegador, al recibir la respuesta 401 con el encabezado WWW-Authenticate, muestra un cuadro de diálogo al usuario pidiéndole un nombre de usuario y una contraseña.
- **Codificación de las credenciales:** una vez que el usuario ingresa sus credenciales (por ejemplo, usuario:contraseña), el navegador las concatena en una sola cadena (usuario:contraseña) y luego las codifica utilizando Base64. La codificación Base64 simplemente convierte la cadena de texto en una serie de caracteres ASCII imprimibles, sin aplicar ningún tipo de cifrado. Por ejemplo, usuario:contraseña se convierte en una cadena como dXN1YXJpbzpwYXNzd29yZA==.

© Universidad Internacional de La Rioja (UNIR)

- **Envío de la solicitud con credenciales:** el navegador vuelve a enviar la solicitud original, pero esta vez incluye un nuevo encabezado Authorization: Basic dXN1YXJpbzpwYXNzd29yZA==, donde la cadena codificada es la que se generó en el paso anterior.

- **Verificación del servidor:** el servidor recibe la solicitud, decodifica la cadena Base64, extrae el nombre de usuario y la contraseña, y los compara con los valores almacenados en su base de datos.

- **Acceso al recurso:** si las credenciales coinciden, el servidor procesa la solicitud y devuelve el recurso solicitado con un código de estado 200 OK. Si las credenciales son incorrectas, el servidor vuelve a responder con un 401 Unauthorized y el proceso se repite.

Riesgos de seguridad

- **Credenciales expuestas:** dado que las credenciales se envían codificadas en lugar de cifradas, un atacante que intercepte el tráfico de red puede decodificarlas fácilmente para obtener el nombre de usuario y la contraseña. Por esta razón, HTTP *basic* solo debe usarse en conexiones cifradas con HTTPS.

- **Ausencia de cierre de sesión:** no existe un mecanismo estándar para que un cliente cierre la sesión. Las credenciales se envían en cada solicitud, lo que hace que un simple *logout* en el cliente sea inefectivo si un atacante ya ha capturado las credenciales.

Por todo lo anteriormente expuesto, la autenticación *basic* es muy insegura. Puede ser objeto de ataques *man in the middle* mediante ARP *spoofing* y redirección de la comunicación a un HTTP-*proxy* interceptador. Además, puede sufrir ataques de repetición una vez capturada una petición mediante *sniffering.* Hay que proteger la comunicación con TLS para evitar ataques de repetición y exposiciones inseguras. Codificar las credenciales en base 64 no es cifrar y son **fácilmente decodificables.**

© Universidad Internacional de La Rioja (UNIR)

Si un servidor web está configurado con autenticación básica, responde con un mensaje HTTP como el de la Figura 3.

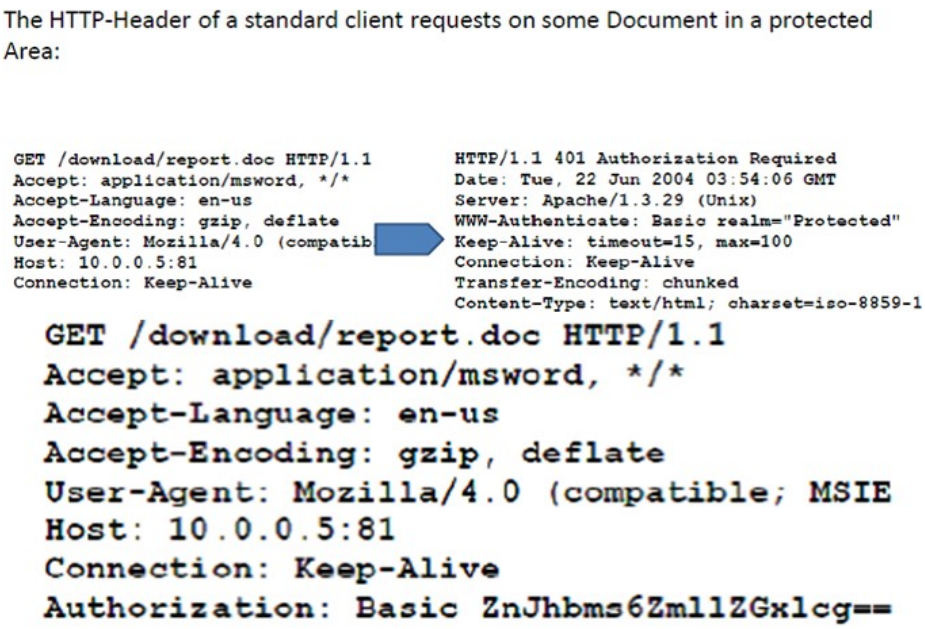


Figura 3. Proceso autenticación básica. Fuente elaboración propia.

Autenticación HTTP *digest*

HTTP *digest* (RFC2617) es una mejora sobre HTTP *basic,* diseñado para **evitar** el envío de la **contraseña** en **texto plano.** En lugar de transmitir la contraseña, se genera un *hash* criptográfico que se envía al servidor para su verificación.

© Universidad Internacional de La Rioja (UNIR)

El **proceso** se puede resumir en los siguientes pasos:

- **Solicitud inicial del cliente:** el cliente (navegador) intenta acceder a un recurso protegido.

- **Respuesta del servidor:** el servidor responde con un código 401 Unauthorized. En la respuesta, incluye un encabezado WWW-Authenticate que contiene varios parámetros clave, como el *realm* (el área de seguridad a la que pertenece el recurso) y un *nonce* (un valor único, aleatorio y de un solo uso). Este *nonce* es crucial para prevenir ataques de *replay.*

- **Cálculo del *hash* por el cliente:** el cliente utiliza los valores proporcionados por el servidor (*realm* y *nonce*) junto con el nombre de usuario y la contraseña para calcular un *hash.* La fórmula es compleja, pero, en esencia, se crea un *hash* de la contraseña y se combina con otros elementos (como el método HTTP, la URI solicitada, el *nonce,* etc.) para generar una cadena final de autenticación.

- **Envío de la respuesta del cliente:** el cliente envía una nueva solicitud al servidor, esta vez incluyendo el encabezado Authorization. Este encabezado contiene el nombre de usuario, el *realm,* el *nonce,* la URI y el *response* (el *hash* calculado en el paso anterior).

- **Verificación del servidor:** el servidor utiliza la misma información (su *nonce,* el nombre de usuario, la contraseña almacenada, la URI, etc.) para realizar el mismo cálculo de *hash.* Si el *hash* que calcula coincide con el *response* enviado por el cliente, la autenticación es exitosa y se permite el acceso al recurso solicitado.

Si un servidor web está configurado con autenticación *digest,* entonces responde con un mensaje HTTP como el de la Figura 4.

© Universidad Internacional de La Rioja (UNIR)

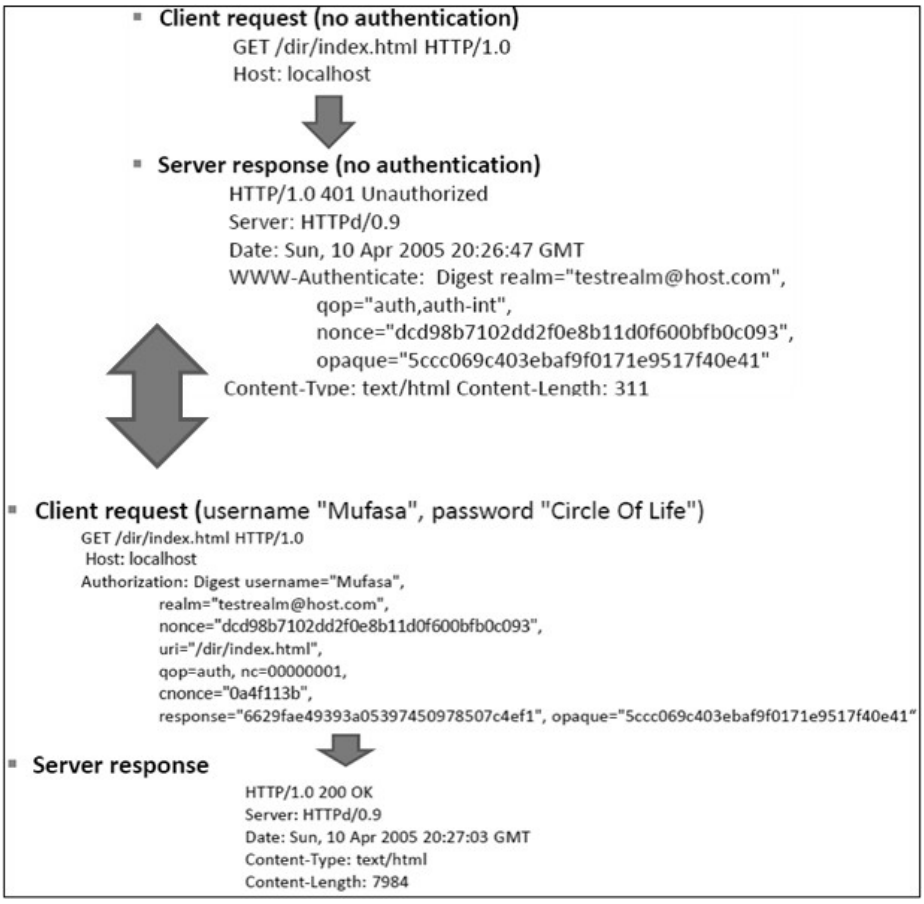


Figura 4. Proceso autenticación básica. Fuente elaboración propia

Puntos clave:

- **Seguridad:** a diferencia de HTTP *basic,* la contraseña se transmite usando concatenación de parámetros y firma *hash* MD5.
- **Protección contra ataques de *replay:*** el uso de un *nonce* único para cada desafío garantiza que un atacante no pueda simplemente capturar el *hash* de una solicitud anterior y reutilizarlo para obtener acceso.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Vulnerabilidades:** aunque es más seguro que HTTP *basic,* no es invulnerable. Por ejemplo, si el servidor se ve comprometido y se obtienen los *hashes* de las contraseñas, un atacante podría autenticarse. Por esta razón, el método ha sido reemplazado, en gran medida, por esquemas de autenticación más robustos.

Al igual que con la autenticación *basic,* se debe emplear el protocolo TLS para **proteger** la **comunicación.** La seguridad del método *digest* depende de la implementación de creación de los **parámetros** CNONCE, NONCE de la cabecera HTTP. Al contrario que la autenticación *basic,* que utiliza codificación Base64 fácilmente reversible, *digest* usa una función *hash,* aunque los valores NONCE deben ser de **un solo uso** o tener un **tiempo muy corto de caducidad** para prevenir ataques de repetición.

© Universidad Internacional de La Rioja (UNIR)

4.3. Autenticación integrada en dominio

Autenticación NTLMv2

Es un **sistema de autenticación** configurable en sistemas operativos Windows, activando el servicio *active directory,* o en sistemas operativos Linux, habilitando el servicio *open ldap.* Es un método diseñado para autenticación en un dominio en **Intranets,** no siendo común en Internet. En este esquema de implementación, las credenciales residen en el **repositorio** del **directorio activo** que hay que proteger, a su vez, de accesos indebidos.

El proceso de autenticación NTLMv2 implica una serie de **intercambios** entre el cliente (el equipo que solicita acceso), el servidor (el recurso al que se accede) y, a menudo, un controlador de dominio (DC) que verifica las credenciales.

El flujo consta de **tres mensajes principales** (ver Figuras 5 y 6):

- **NEGOTIATE_MESSAGE (Tipo 1):** el cliente inicia el proceso enviando este mensaje al servidor. Contiene información sobre las capacidades de autenticación del cliente, como las versiones de NTLM que soporta. Es una especie de saludo inicial.
- **CHALLENGE_MESSAGE (Tipo 2):** el servidor responde al cliente con un mensaje de desafío. Este mensaje contiene:
  - **Un desafío del servidor** *(server challenge):* un número aleatorio de 8 bytes.
  - **Información del servidor:** como su nombre de dominio y nombre de *host.*

© Universidad Internacional de La Rioja (UNIR)

- **AUTHENTICATE_MESSAGE (Tipo 3):** el cliente recibe el desafío del servidor y lo utiliza, junto con otros elementos, para calcular una respuesta. Aquí es donde NTLMv2 difiere significativamente de NTLMv1. El cliente no solo usa el desafío del servidor, sino que también genera un desafío del cliente *(client challenge)* de longitud variable que incluye:

  - Un valor aleatorio.
  - Una marca de tiempo.
  - Información del dominio y el usuario.

El cliente usa la contraseña del usuario (en su formato de *hash* NTLM) y estos elementos para generar el *hash* NTLMv2. La fórmula para el *response* (NTLMv2 *hash*) es compleja, pero se basa en el **algoritmo HMAC-MD5.** El cliente envía este *hash* al servidor en el mensaje AUTHENTICATE_MESSAGE, junto con el nombre de usuario y el dominio.

Verificación del servidor: el servidor recibe el *response* del cliente y, si se trata de un controlador de dominio, lo verifica directamente con la información de la cuenta del usuario. Si no, el servidor reenvía el desafío del servidor, el *response* del cliente y el nombre de usuario al controlador de dominio. El DC, que tiene el *hash* de la contraseña del usuario, realiza el mismo cálculo de *hash* NTLMv2. Si el *hash* que calcula coincide con el *response* enviado por el cliente, la autenticación es exitosa y se concede el acceso.

Características de **seguridad clave** de NTLMv2:

- **Uso de *client challenge:*** la inclusión de un desafío generado por el cliente y una marca de tiempo en el cálculo del *hash* previene los ataques de retransmisión (donde un atacante captura y reutiliza el *hash* de una sesión anterior).

© Universidad Internacional de La Rioja (UNIR)

- ***Hash* HMAC-MD5:** el uso de este algoritmo criptográfico es más seguro que el utilizado en NTLMv1, lo que hace mucho más difícil para un atacante descifrar el *hash* de la contraseña original.

- **Vulnerabilidades residuales:** aunque NTLMv2 es mucho más robusto que NTLMv1, sigue siendo vulnerable a ataques, como la retransmisión de NTLM, repetición activa y los ataques de *pass-the-hash* si un atacante logra obtener el *hash* NTLM del usuario. Por esta razón, Microsoft recomienda encarecidamente el uso de Kerberos en entornos de *active directory,* ya que es un protocolo más moderno y seguro.

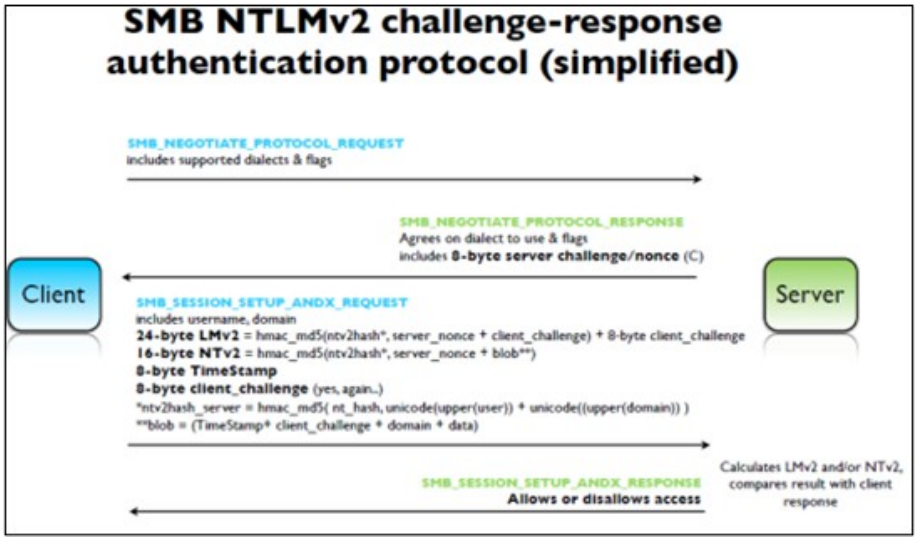


Figura 5. Autenticación NTLMv2. Fuente: Ochoa y Azubel, 2010.

© Universidad Internacional de La Rioja (UNIR)

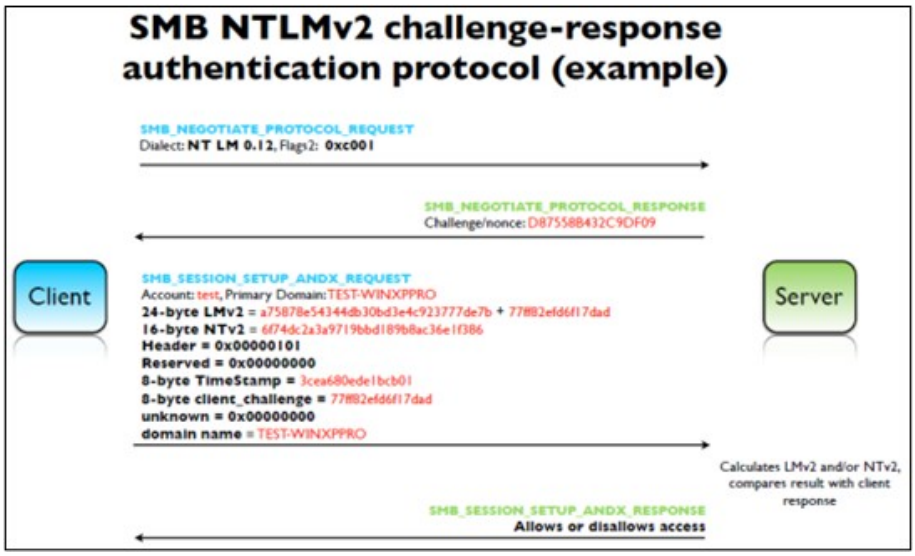


Figura 6. Autenticación NTLMv2. Fuente: Ochoa y Azubel, 2010.

Ataques que puede sufrir el **protocolo NTLMv2:**

- *Pass the hash.*
- Ataques de repetición activos.
- NTLM *relay.*

A continuación, se presenta la secuencia de un **ataque de repetición activo** debido a que el protocolo repite los desafíos periódicamente:

En un primer paso, el atacante realiza intentos de **conexión** y obtiene **valores NONCE** que guarda para utilizar después.

El atacante hace que el usuario se conecte a él mediante el envío de un *email* que contiene un **enlace** a su sitio web. El usuario se conecta al servidor del atacante y este le envía al usuario los **desafíos obtenidos** en el primer paso y anota las respuestas.

© Universidad Internacional de La Rioja (UNIR)

El atacante intenta conectarse a la máquina del usuario esperando que se repita alguno de los desafíos obtenidos en el paso uno, cuyas respuestas obtuvo en el paso 2. Cuando se repite un desafío, el atacante estará en condiciones de **autenticarse** en la máquina del usuario porque tiene la respuesta.

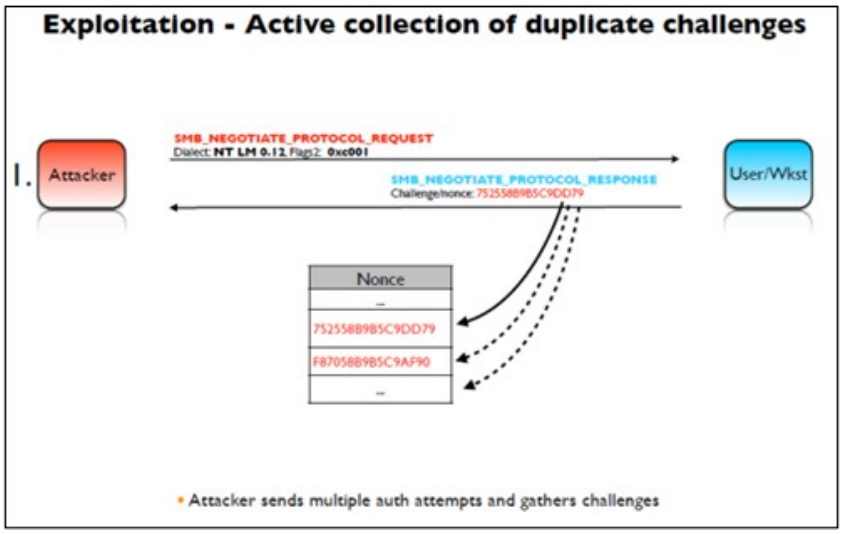


Figura 7. Autenticación NTLMv2. Ataque paso 1. Fuente: Ochoa y Azubel, 2010.

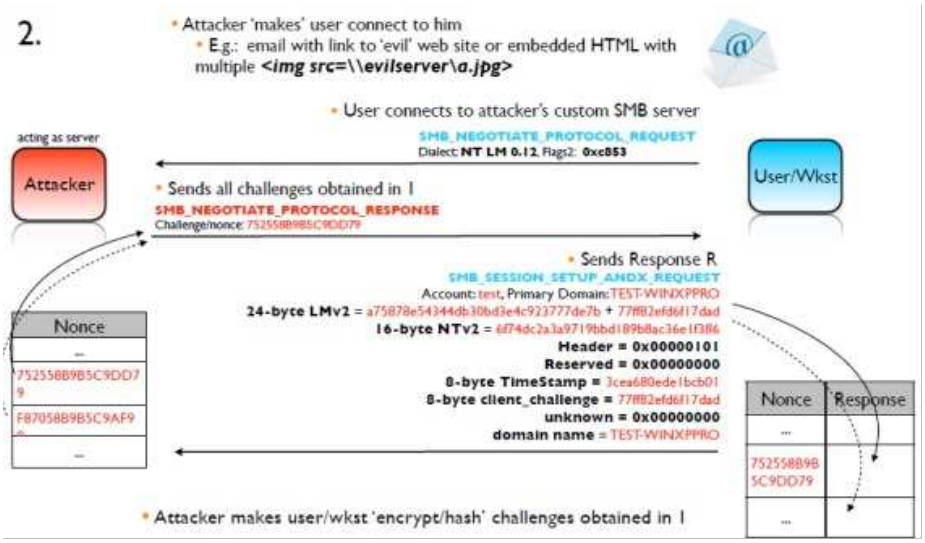


Figura 8. Autenticación NTLMv2. Ataque paso 2. Fuente: Ochoa y Azubel, 2010.

© Universidad Internacional de La Rioja (UNIR)

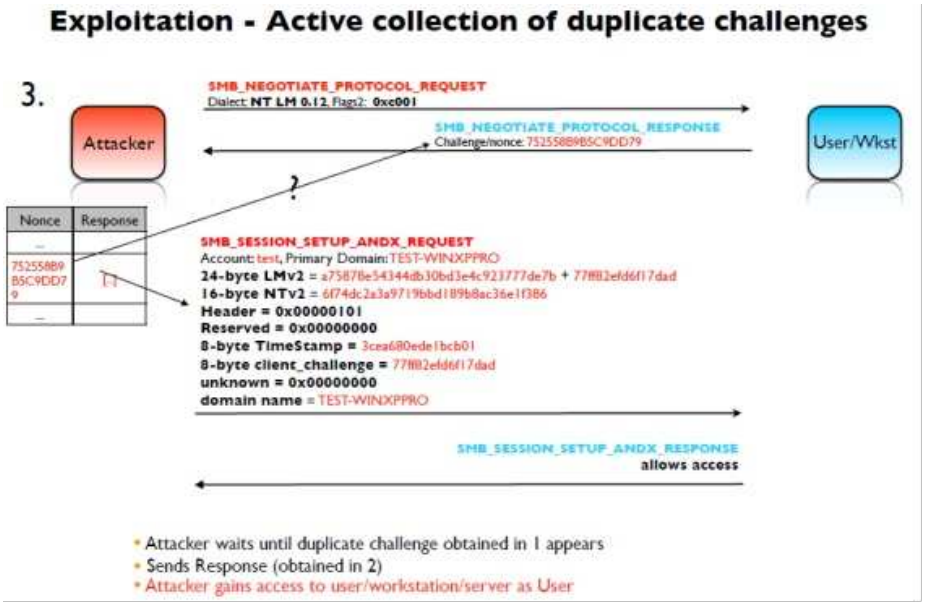


Figura 9. Autenticación NTLMv2. Ataque paso 3. Fuente: Ochoa y Azubel, 2010.

Autenticación Kerberos

Kerberos (Ganesh, 2019) es un protocolo que maneja el concepto de TGT (tique para solicitar servicios) y de tique de servicio para **acceder** a los **servicios.** El protocolo Kerberos es un sistema de **autenticación** de red que utiliza **criptografía** de **clave secreta** para asegurar la comunicación entre un cliente y un servidor a través de una red no segura. Se basa en un tercero de confianza, el centro de distribución de claves (KDC), para verificar la **identidad** de los usuarios y servicios sin enviar contraseñas por la red.

© Universidad Internacional de La Rioja (UNIR)

El **proceso** funciona en varios pasos:

- **Solicitud de *ticket-granting ticket*** (TGT) (Figura 10): cuando un usuario inicia sesión, el cliente (su ordenador) envía una solicitud de autenticación al servidor de autenticación (AS), que es parte del KDC. El AS verifica las credenciales del usuario (sin que la contraseña se envíe en texto plano) (1) y, si son correctas, le emite un TGT *(ticket-granting ticket)* (2). Este TGT está cifrado con la clave secreta del servidor de concesión de tiques (TGS) y también contiene una clave de sesión que se comparte con el cliente.

- **Solicitud de *ticket* de servicio:** cuando el cliente necesita acceder a un servicio, envía el TGT (que no puede leer) y una solicitud de tique de servicio al TGS (3). El TGS descifra el TGT con su clave secreta, verifica la identidad del cliente y, si todo es válido, emite un tique de servicio para el servicio solicitado (4). Este tique contiene la clave de sesión compartida entre el cliente y el TGS y está cifrado con la clave secreta del servidor de destino.

- **Acceso al servicio:** el cliente recibe el tique de servicio y lo envía al servidor que quiere acceder (5). El servidor utiliza su clave secreta para descifrar el tique, obtener la clave de sesión compartida y la información del cliente. Esto le permite verificar la identidad del cliente y establecer un canal de comunicación seguro y autenticado (6).

© Universidad Internacional de La Rioja (UNIR)

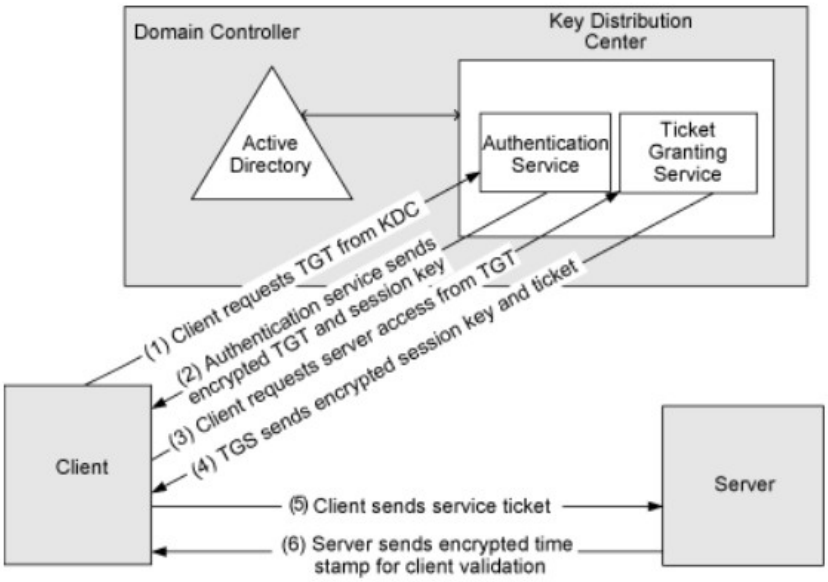


Figura 10. Autenticación Kerberos. Fuente: Alarcón, 2015.

Ataques a Kerberos

Aunque Kerberos es un protocolo de **autenticación** robusto, no es invulnerable. Los ataques más comunes se centran en la **manipulación** de sus tiques y en el compromiso de la infraestructura, especialmente el centro de distribución de claves (KDC). Ataques basados en el sistema Kerberos (Motero, 2021):

- ***Golden ticket****:* este es uno de los ataques más peligrosos. Un atacante que logra comprometer el servidor krbtgt del KDC (la cuenta maestra utilizada para firmar los tiques) puede forjar un *golden ticket.* Este tique permite al atacante obtener acceso ilimitado a cualquier recurso en el dominio, ya que parece ser emitido por el propio KDC. El ataque otorga persistencia y control total.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- ***Silver ticket:*** similar al *golden ticket,* pero más limitado. En este caso, el atacante forja un tique para acceder a un servicio específico (por ejemplo, un servidor de archivos o una base de datos) en lugar de a todo el dominio. El atacante necesita obtener el *hash* de la cuenta de servicio, lo que a menudo es más fácil que comprometer el KDC.

- **Kerberoasting**: este ataque se enfoca en las cuentas de servicio (SPN o Service Principal Name). Un atacante con credenciales de un usuario de dominio puede solicitar un tique de servicio para una cuenta de servicio. El KDC devuelve este tique cifrado con el *hash* de la contraseña de la cuenta de servicio. El atacante puede capturar este tique *offline* y usar herramientas de fuerza bruta para descifrar el *hash* de la contraseña de la cuenta de servicio.

- **AS-REP Roasting**: este ataque explota una configuración vulnerable en las cuentas de usuario *(do not require* Kerberos *preauthentication).* Si esta opción está deshabilitada, un atacante puede solicitar un tique de autenticación inicial (AS-REQ) para un usuario sin proporcionar la preautenticación (generalmente, la contraseña). El KDC responde con un *hash* que el atacante puede capturar y descifrar *offline* para obtener la contraseña del usuario.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Ataques de **suplantación** y **retransmisión:**

- ***Pass-the-ticket*** (PtT): este ataque permite a un atacante autenticarse en un servicio utilizando un tique Kerberos válido sin necesidad de conocer la contraseña del usuario. Si un atacante compromete un sistema y obtiene el tique de un usuario legítimo (por ejemplo, un administrador de dominio), puede usarlo para suplantar a ese usuario y acceder a otros recursos de la red.

- ***Pass-the-hash*** (PtH): aunque Kerberos utiliza tiques, muchos sistemas Windows todavía almacenan *hashes* de contraseñas para compatibilidad con NTLM. Si un atacante roba el *hash* de una contraseña, puede usarlo para generar un tique Kerberos válido sin conocer la contraseña original.

Recomendaciones Kerberos:

- Habilitar una **política fuerte** de **contraseñas.** Evitar el uso de cuentas sin preautenticación.

- **Evitar** la **ejecución de servicios** en el contexto de cuentas de usuario.

- **Deshabilitar** los **algoritmos de cifrado débiles** de Kerberos (solo se deberían permitir cifrado con AES (256)).

© Universidad Internacional de La Rioja (UNIR)

4.4. Autenticación web

Autenticación basada en *token* JWT

La autenticación basada en token es un enfoque moderno que **elimina** la necesidad de **sesiones** en el servidor, haciéndolo ideal para arquitecturas de microservicios y API. El método más popular es JSON Web Token (JWT).

El **proceso** es el siguiente:

- **Inicio de sesión:** el usuario envía sus credenciales al servidor.
- **Generación del *token:*** el servidor verifica las credenciales y, si son correctas, genera un JWT. Este *token* contiene información sobre el usuario (los llamados *claims*) y está firmado digitalmente para asegurar su integridad.
- **Envío del *token:*** el servidor envía el JWT al cliente. El cliente lo almacena (por ejemplo, en el *localStorage* o en una *cookie*).
- **Peticiones posteriores:** en cada solicitud, el cliente envía el JWT en el encabezado Authorization.

Validación: el servidor verifica la firma del *token* para asegurar que no ha sido alterado y extrae la información del usuario para permitir el acceso.

Ventajas: es sin estado *(stateless),* lo que significa que el servidor no necesita almacenar información de sesión. Esto lo hace muy escalable. Los *tokens* son portables y pueden ser usados por diferentes servicios.

Desventajas: la seguridad depende de la protección del *token* en el lado del cliente. Si un atacante roba el *token,* puede usarlo hasta que expire. La revocación de un *token* antes de su expiración puede ser compleja.

© Universidad Internacional de La Rioja (UNIR)

El *token* JWT consta de **tres partes** separadas por puntos y codificadas en base64 (codificación!= encriptación): el ***header*,** el ***payload*** o *claims* y la ***signature***. Veamos qué significa cada uno:

***String* completo:**

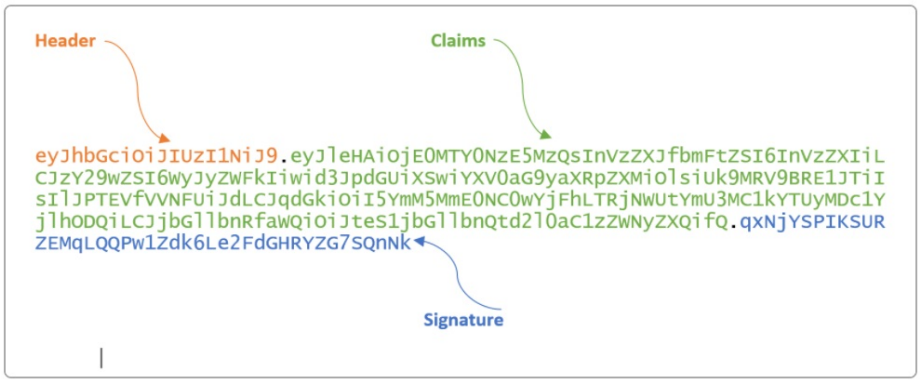


Figura 11. JWT. Fuente elaboración propia.

El ***header*** es la cabecera del *token* que, a su vez, tiene otras dos partes, el **tipo**, en este caso un JWT, y la **codificación** utilizada.

```
eyJhbGciOiJIUzI1NiJ9
```

Figura 12. Ejemplo. Fuente elaboración propia.

```
{
 "alg":"HS256",
 "typ":"JWT"
}
```

El ***payload*** está compuesto por los llamados JWT *claims,* donde irán colocados los **atributos** que definen el *token.*

- **User_name:** identifica el sujeto del *token,* por ejemplo, un identificador de usuario.

© Universidad Internacional de La Rioja (UNIR)

- **Iat:** identifica la fecha de creación del *token* (válido para si queremos ponerle una fecha de caducidad) en formato de tiempo UNIX.
- **Exp:** identifica a la fecha de expiración del *token.* Podemos calcularla a partir del Iat. También en formato de tiempo UNIX.
- **Jti:***token* de autenticación.
- **Otros.**

Estos atributos pueden **variar,** incluir información de **roles,** etc. Por ejemplo:


```
{
 "exp": 1416471934,
 "user_name": "user",
 "scope": [
 "read",
 "write"
],
 "authorities": [
 "ROLE_ADMIN",
 "ROLE_USER"
],
 "jti": "9bc92a44-0b1a-4c5e-be70-da52075b9a84",
 "client_id": "my-client-with-secret"
}
```

```
eyJleHAiOiAxNDE2NDcxOTM0LC|J1c2VyX25hbWUiOiAidXNlciIsInNjb3BlIjpbInJlYWQiL
CJ3cmI0ZSJdLCJhdXRob3JpdGllcyI6WyJST0xFX0FETUlOIiwiUk9MRV9VU0VSII0sImp0aSI
6ICI5YmM5MmE0NC0wYjFhLTRjNWUtYmU3MC1kYTUyMDc1YjlhODQiLCJjbGllbnRfaWQiOiJte
S1jbGllbnQtd2l0aC1zZWNyZXQifQ
```


© Universidad Internacional de La Rioja (UNIR)

La firma, ***signature***, es la tercera y última parte del JSON web *token.* Está formada por los anteriores componentes, *header* y *payload,* codificados en Base64, y firmada con una **clave secreta** que (HMAC-SHA256) debe estar almacenada en el *backend.* Así, sirve de *hash* para verificar la **integridad** del *token.* En el ejemplo sería:

```
qxNjYSPIKSURZEMqLQQPw1Zdk6Le2FdGHRYZG7SQnNk
```

Figura 13. Ejemplo. Fuente: elaboración propia.

Para una implementación correcta que garantice la **confidencialidad, integridad** y **fortaleza** ante ataques de repetición, debe usar valor NONCE en la solicitud del *token* y algoritmo RSA256, en lugar de HSA256 (Solicitar un token de autenticación, s. f.).

© Universidad Internacional de La Rioja (UNIR)

Vulnerabilidades y ataques comunes en la autenticación basada en JWT (JSON web *token*)

A continuación, se resumen las principales **vulnerabilidades** y ataques asociados a la autenticación basada en JWT junto con sus respectivas recomendaciones de **seguridad.**

- **Falsificación de firma:**
  - **Vulnerabilidad:** si el algoritmo de firma de un JWT está mal configurado, un atacante podría modificar el *token* y generar una firma falsa. Por ejemplo, si un JWT se firma usando el algoritmo " none " (sin firma), un atacante podría crear un *token* sin firma y pasarlo como válido.
  - **Recomendación:** deshabilitar el algoritmo " none " (asegúrese de que el servidor siempre verifique la firma del *token* con un algoritmo seguro, como RS256, y use una clave privada para firmar y una clave pública para verificar) y validar la firma de los JWT siempre (asegúrese de que el *token* sea firmado por la fuente legítima antes de confiar en su contenido).
- **Uso de algoritmos débiles:**
  - **Vulnerabilidad:** el uso de algoritmos de firma débiles, como HS256 (HMAC con SHA-256) con una clave secreta débil o fácil de adivinar, puede permitir que un atacante genere un JWT válido. Aunque RS256 también es susceptible de ser mal implementado (si no se gestionan bien las claves), HS256 se considera especialmente vulnerable si la clave secreta no es robusta.
  - **Recomendación:** usar algoritmos más seguros, como RS256 o ES256 (firmado con clave pública/privada), en lugar de HS256, y asegurarse de que las claves privadas y secretas estén protegidas adecuadamente y sean lo suficientemente complejas.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Robo de claves privadas:**

  - **Vulnerabilidad:** si la clave privada utilizada para firmar los JWT se ve comprometida, un atacante puede generar *token* válidos que podrían usarse para suplantar a un usuario u obtener acceso no autorizado.

  - **Recomendación:** proteger las claves privadas. Utilizar un entorno seguro para almacenar las claves privadas, como un almacén de claves (HSM, Key Vault), y asegurarse de que las claves no estén expuestas en el código fuente o repositorios. Además, implementar un mecanismo de rotación de claves para cambiarlas periódicamente.

- **Reutilización de *token:***

  - **Vulnerabilidad:** un atacante que consiga un JWT válido podría reutilizarlo para acceder a un recurso protegido, incluso si la víctima ya ha cerrado sesión.

  - **Recomendación:** utilice JWT de corta duración (con un tiempo de vida limitado) y combine con *refresh token* para renovar *tokens* sin que los usuarios se vean obligados a iniciar sesión cada vez. Aplique verificación de IP o verificación de agente de usuario *(User-Agent)* como medida adicional para reducir el riesgo de que un *token* capturado sea usado en otro dispositivo.

- **Exposición de información sensible:**

  - **Vulnerabilidad:** Los JWT suelen ser enviados a través de HTTP o en las *cookies.* Si no están cifrados, pueden exponer información sensible (como identificadores de usuario, roles o permisos) a través de ataques de interceptación *(man-in-the-middle).*

  - **Recomendación:** siempre use HTTPS para enviar JWT. La información en los *tokens* puede ser decodificada fácilmente si se intercepta, aunque no se modifique. Cifre el contenido sensible dentro del JWT utilizando un cifrado (JWE, JSON Web Encryption) si el *token* contiene datos sensibles.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Fugas de *tokens* en almacenamiento no seguro:**

  - **Vulnerabilidad:** almacenamiento de JWT en lugares inseguros, como *localStorage* o *sessionStorage,* puede exponerlos a ataques de *cross-site scripting* (XSS). Los *tokens* pueden ser robados por un atacante si puede ejecutar *scripts* en el navegador del usuario.

  - **Recomendación:** almacene JWT en *cookies* seguras con la opción HttpOnly activada, lo que impide el acceso desde JavaScript. Asegúrese de que las *cookies* estén marcadas como *secure* para solo ser enviadas a través de HTTPS. Y, por último, evite el almacenamiento en *localStorage* o *sessionStorage,* cuando sea posible, debido a los riesgos de XSS.

- **Falta de validación de la expiración:**

  - **Vulnerabilidad:** si un JWT no tiene una fecha de expiración adecuada (exp), o si no se valida correctamente, un atacante podría seguir usando un *token* incluso después de que haya caducado.

  - Recomendación: implementar y validar siempre el campo exp (expiración) del JWT para garantizar que los *tokens* caducados no sean aceptados. Revocar tokens cuando sea necesario. Aunque no siempre es sencillo con JWT, considere tener un mecanismo para revocar *token,* como una lista negra de JWT expirados o comprometidos.

© Universidad Internacional de La Rioja (UNIR)

- Falta de validación del *issuer* (iss):

  - **Vulnerabilidad:** un atacante podría generar un JWT falso que pase por válido si el servidor no valida correctamente el campo iss *(issuer),* que indica la fuente del *token.* Si no se valida el emisor correctamente, un *token* de otro servicio podría ser aceptado.

  - Recomendación: validar siempre el campo iss *(issuer)* del JWT. Asegúrese de que el *token* provenga de una fuente legítima y confiable. También se puede verificar el campo aud *(audience)* para asegurarse de que el *token* es válido para la aplicación o servicio al que se está accediendo.

- **Manipulación de *claims:***

  - **Vulnerabilidad:** dado que el contenido de un JWT es solo firmado y no cifrado (a menos que se utilice JWE), un atacante podría manipular los *claims* del JWT, como los roles o permisos del usuario, y enviar el *token* modificado al servidor.

  - Recomendación: Validar y verificar los *claims* en el servidor antes de confiar en cualquier información contenida en el JWT (por ejemplo, roles, ID de usuario, etc.). Aplique una política estricta de *least privilege* (mínimos privilegios) para los permisos basados en JWT.

Recomendaciones generales de seguridad para autenticación basada en JWT

- **UtilizarHTTPS** en todo momento para proteger los JWT durante la transmisión.

- Implementar **rotación de claves** para mitigar el riesgo de que una clave privada sea comprometida.

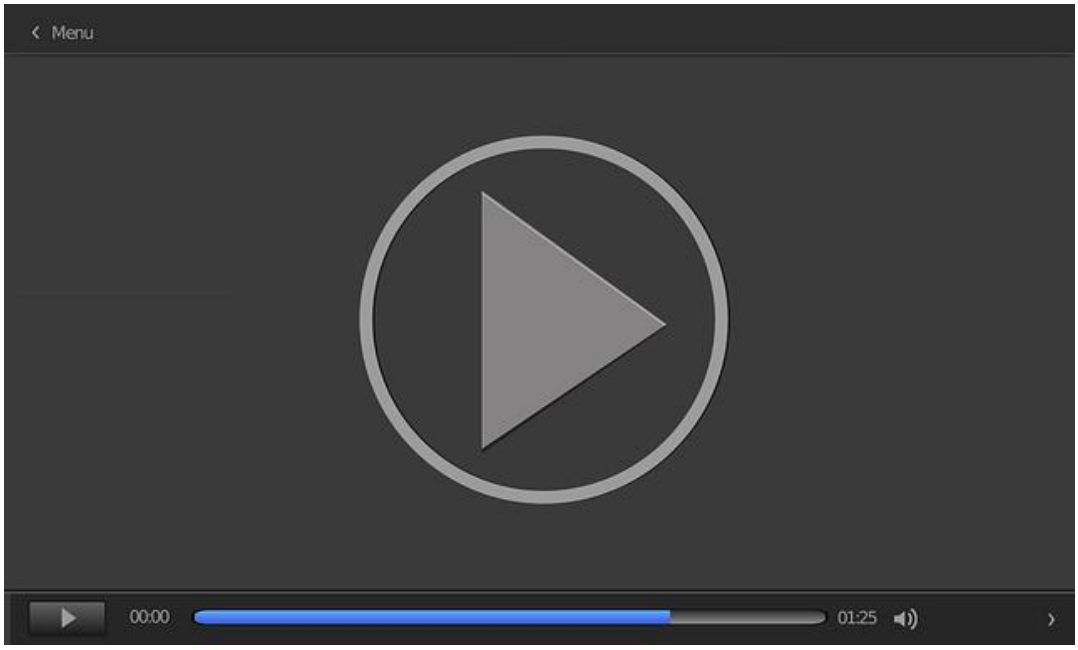
- Usar JWT de **corta duración** y combinarlos con *refresh tokens* para mayor seguridad.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Verificar** y **validar** todos los campos importantes (exp, iss, aud) de un JWT antes de confiar en su contenido.
- Evitar **almacenamiento inseguro** de *tokens* en el cliente *(localStorage, sessionStorage).* Preferiblemente, use *cookies* seguras con HttpOnly y Secure.
- Realizar **auditoría continua** y **monitoreo** de los JWT en uso para detectar posibles ataques y vulnerabilidades.

Se recomienda visualizar el vídeo *JSON Web Token* para saber más sobre el funcionamiento de la autenticación basada en2 JWT.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=cc5a5436-0f66-4574-ac22-ae5d00af8728

© Universidad Internacional de La Rioja (UNIR)

Autenticación: mTLS

mTLS, o mutual TLS, es un método de **autenticación mutua** que se usa para asegurar las comunicaciones entre un cliente y un servidor. A diferencia del TLS *(transport layer security)* estándar, donde solo el servidor se autentica ante el cliente, con mTLS ambas partes se autentican entre sí usando **certificados digitales.**

¿Cómo funciona mTLS? El proceso de autenticación mutua en mTLS ocurre en varios **pasos clave** durante el *handshake* TLS:

- **El cliente se conecta al servidor:** el cliente inicia una conexión TLS y envía un mensaje ClientHello con información sobre las suites de cifrado que soporta.
- **El servidor responde y pide un certificado:** el servidor responde con un ServerHello su propio certificado digital y, crucialmente, un mensaje CertificateRequest pidiendo el certificado del cliente.
- **El cliente envía su certificado:** el cliente verifica el certificado del servidor y, si es válido, responde con su propio certificado y una firma digital para probar que realmente lo posee.
- **El servidor verifica al cliente:** el servidor valida el certificado del cliente y su firma. Si todo es correcto, ambas partes pueden completar el *handshake* y establecer una conexión segura y cifrada.

© Universidad Internacional de La Rioja (UNIR)

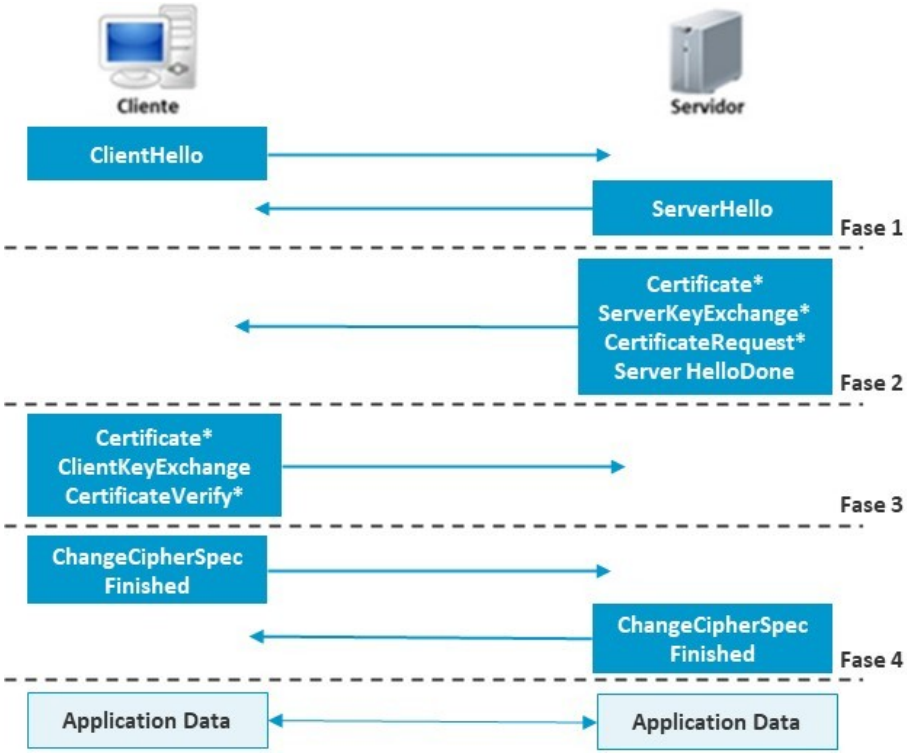


Figure 13. TLS *handshake protocol.* Fuente: elaboración propia.

En ocasiones, se protege solamente en la autenticación y no durante el resto de la sesión. Esto da lugar a ataques de **secuestro de sesión.** Una herramienta que se puede utilizar para captura de sesiones inseguras es Firesheep (Butler, 2010). Otra herramienta, como BlackSheep (Sobrier, 2010), sirve para detectar Firesheep en Firefox instalada como una *add extension*. Conclusión: **se debe cifrar toda la sesión.**

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Hay varios **ataques potenciales** que pueden comprometer una conexión mTLS:

- **Ataques a la autoridad de certificación (CA):** el mayor riesgo para mTLS no está en el protocolo, sino en la confianza en la cadena de certificados.
  - **Compromiso de la CA:** un atacante que logra comprometer una CA puede emitir certificados digitales fraudulentos para clientes o servidores. Con un certificado falso pero válido, el atacante puede hacerse pasar por una entidad de confianza y establecer una conexión mTLS con la víctima.
  - **Ataques de *man-in-the-middle* (MITM):** aunque mTLS está diseñado para prevenir ataques MITM, un atacante que posee un certificado emitido por una CA de confianza puede interceptar y descifrar la comunicación. El cliente confiará en el certificado del atacante porque fue firmado por una CA que considera legítima.
- **Ataques a las claves y certificados:** estos ataques se centran en el robo o el mal uso de los certificados de las propias entidades.
  - **Robo de claves privadas:** si un atacante roba la clave privada de un cliente o servidor, puede clonar el certificado y hacerse pasar por esa entidad. Esto le permite establecer conexiones mTLS no autorizadas.
  - **Configuración incorrecta:** una mala configuración del servidor, como no verificar la revocación del certificado del cliente (OCSP o CRL), puede permitir que un atacante utilice un certificado de cliente revocado para establecer una conexión.

© Universidad Internacional de La Rioja (UNIR)

- **Ataques a los puntos finales *(endpoints):*** estos ataques no se dirigen al protocolo en sí, sino a las vulnerabilidades en los sistemas que lo utilizan.

- **Ataques de denegación de servicio (DoS):** un atacante puede intentar abrumar al servidor con una gran cantidad de solicitudes de *handshake* mTLS, lo que consume recursos de CPU y memoria. Aunque no compromete la seguridad, puede hacer que el servicio no esté disponible para los usuarios legítimos. La renegociación TLS iniciada por el cliente puede dar lugar a ataques de denegación de servicio DoS. Las conexiones TLS con renegociación requieren de 10-35x más de rendimiento en el servidor. La solución es *reverse* TLS (Bernat, 2011) para que el cliente se ocupe de más tareas del protocolo. Otras técnicas de mitigación pasan por: deshabilitar la renegociación TLS, limitar el tiempo de *handshakes* TLS, incrementar la capacidad de proceso del servidor y aumentar la carga en el cliente.

Recomendaciones de **seguridad** del protocolo TLS:

- **Usar siempre la última versión de TLS:** la versión más actual es TLS 1.3. Las versiones anteriores (1.2 y, en particular, 1.1 y 1.0) tienen vulnerabilidades conocidas y ya no son consideradas seguras. Al implementar TLS, configura tu servidor para que solo acepte conexiones con TLS 1.3, si es posible.

- **Seleccionar conjuntos de cifrado fuertes:** RSA 2048, AES 256.

- **Habilitar HSTS (HTTP *strict transport security*):** esta cabecera HTTP le dice a los navegadores que solo se conecten a tu sitio usando HTTPS. Esto ayuda a prevenir ataques que intentan degradar la conexión a HTTP.

© Universidad Internacional de La Rioja (UNIR)

Autenticación basada en formularios

La autenticación basada en formularios **funciona de la siguiente manera**:

- **Inicio de sesión:** el usuario envía sus credenciales al servidor.
- **Verificación:** el servidor valida las credenciales y, si son correctas, crea una sesión única para ese usuario.
- **Creación de *cookie:*** el servidor almacena la información de la sesión en su base de datos y envía un ID de sesión al navegador del usuario, que se guarda en una *cookie.*
- **Peticiones posteriores:** en cada solicitud subsiguiente, el navegador envía la *cookie* con el ID de sesión.
- **Validación:** el servidor busca el ID de sesión en su base de datos para verificar la identidad del usuario y permitir el acceso a los recursos.

© Universidad Internacional de La Rioja (UNIR)

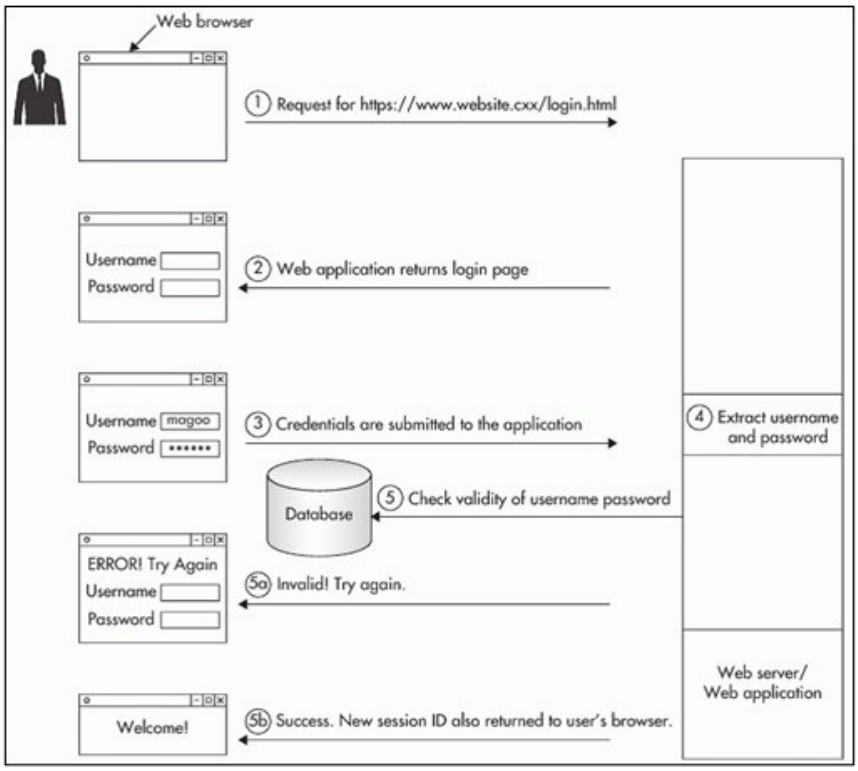


Figura 14. Autenticación desde aplicación web. Fuente: Sullivan, 2012.

```
HTTP/1.1 302 Moved Temporarily
Date: Sun, 17 Apr 2011 18:28:52 GMT
Server: Apache/2.2.15 (Unix) mod_ssl/2.2.15 OpenSSL/1.0.0
Cache-Control: no-cache="Set-Cookie"
Location: https://www.website.cxx/main.jsp
Set-Cookie: JSESSIONID=vPGwNrxDzSshFhd2LJcFqVvYL0s8r3wnzYJ6B5mJMNwJh
0bg3PWN!-1251942355;path=/;secure;httponly
X-Powered-By: Servlet/2.5 JSP/2.1
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1
Content-Length: 437

<html><head><title>302 Moved Temporarily</title></head>
```

Figura 15. Autenticación desde aplicación web y respuesta después de una autenticación válida desde la aplicación. Fuente elaboración propia.

- **Ventajas**: es un método bien establecido y fácil de entender. La información de la sesión se guarda en el servidor, lo que puede ser más seguro.

© Universidad Internacional de La Rioja (UNIR)

- Desventajas:
  - Vulnerable a ataques de **fuerza bruta.**
  - Vulnerable a ataques de ***phishing:*** los atacantes pueden crear páginas de inicio de sesión falsas para engañar a los usuarios y robar sus credenciales.
  - **Gestión de estado:** el manejo de la sesión y las *cookies* (HTTPonly, *secure, samesite*) puede ser complejo y requiere una implementación segura para evitar ataques, como el secuestro de sesión *(session hijacking).*

Recomendaciones de **seguridad:**

- **Nunca almacenar contraseñas en texto plano:** en lugar de guardar las contraseñas directamente, utiliza un *hash* criptográfico robusto (como bcrypt o Argon2).
- **Limitar los intentos de inicio de sesión:** después de un número de intentos fallidos (por ejemplo, 3 a 5), bloquea la cuenta o introduce una demora exponencial en los siguientes intentos. Esto frustra los ataques automatizados.
- **Implementa la autenticación multifactor (MFA):** ofrece a los usuarios la opción de usar un segundo factor de autenticación (como una contraseña de un solo uso enviada por SMS o una aplicación generadora de códigos). Esto protege las cuentas incluso si la contraseña principal ha sido robada.
- **Verificar la actividad sospechosa:** monitorea los inicios de sesión desde nuevas ubicaciones o dispositivos y notifica al usuario si detectas una actividad inusual en su cuenta.
- **Utilizar HTTPS siempre:** todas las comunicaciones, desde el envío del formulario de inicio de sesión hasta el acceso a recursos protegidos, deben realizarse a través de HTTPS. Esto garantiza que los datos de la sesión y las credenciales estén cifrados en tránsito y previene los ataques *man-in-the-middle.*

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

- **Configurar cookies seguras:** las *cookies* de sesión deben tener los siguientes atributos:
  - **HttpOnly:** previene que las *cookies* sean accesibles a través de JavaScript, lo que mitiga los ataques de *cross-site scripting* (XSS).
  - ***Secure:*** asegura que la *cookie* solo se envíe a través de conexiones HTTPS.
  - **SameSite:** previene que los navegadores envíen la *cookie* en solicitudes entre sitios y protege contra los ataques de *cross-site request forgery* (CSRF).
- **Renueva el ID de sesión:** genera un nuevo ID de sesión después de un inicio de sesión exitoso. Esto previene que un atacante utilice un ID de sesión robado de una sesión no autenticada para tomar el control de la cuenta.
- **Valida la entrada del usuario:** asegúrate de que las contraseñas cumplan con una política de complejidad (combinando letras, números y símbolos). Esto ayuda a los usuarios a crear contraseñas fuertes.
- **Sanear toda la entrada del usuario:** antes de procesar cualquier dato, sanéalo para prevenir la inyección de código (por ejemplo, SQL o JavaScript). Esto es especialmente importante en el nombre de usuario.
- **Utilizar *captchas* o *honeypots:*** para proteger los formularios de inicio de sesión de los bot, puedes utilizar *captchas* o campos ocultos *(honeypots)* que solo los bot llenarán.
- **Evitar el autocompletado en campos de contraseña:** esto es una recomendación de seguridad muy importante para evitar que los navegadores almacenen contraseñas de manera insegura.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Autenticación de varios factores

La autenticación multifactor (MFA) o autenticación de dos factores (2FA) es una **capa adicional** de seguridad. Requiere que el usuario proporcione dos o más factores de verificación para acceder a una cuenta.

Los factores de autenticación se dividen en **tres categorías:**

- **Conocimiento:** algo que solo el usuario sabe (ej., contraseña, PIN).
- **Posesión:** algo que solo el usuario tiene (ej., *smartphone* para códigos SMS, *token* de seguridad, aplicación de autenticación).
- **Inherencia:** algo que el usuario es (ej., huella dactilar, reconocimiento facial).

Ventajas: aumenta significativamente la seguridad, ya que incluso si un atacante roba un factor (como la contraseña), no podrá acceder a la cuenta sin el segundo factor.

Desventajas: puede agregar fricción al proceso de inicio de sesión. La implementación puede ser compleja dependiendo de los factores utilizados.

Autenticación sin contraseña (passwordless)

La autenticación sin contraseña busca **eliminar** la **dependencia** de las contraseñas, que son un punto débil común. Los métodos más populares incluyen:

- **Enlaces mágicos** *(magic links):* se envía un enlace único y de un solo uso al correo electrónico del usuario. Hacer clic en él autentica al usuario.

© Universidad Internacional de La Rioja (UNIR)

- **Códigos únicos:** se envía un código temporal por SMS o correo electrónico. El usuario lo introduce para iniciar sesión.

- **WebAuthn:** un estándar de la W3C que permite la autenticación mediante criptografía de clave pública al utilizar dispositivos, como llaves de seguridad o datos biométricos.

Ventajas: mejora la experiencia del usuario y la seguridad al eliminar el riesgo de contraseñas débiles, reutilizadas o robadas.

Desventajas: la autenticación sin contraseña, o *passwordless,* elimina la necesidad de que los usuarios memoricen y escriban una contraseña. En cambio, utiliza otros factores para verificar la identidad, como enlaces mágicos (enviados por correo electrónico), códigos de un solo uso (enviados por SMS) o datos biométricos.

- ***Phishing:*** los atacantes pueden engañar a los usuarios para que hagan clic en un enlace de inicio de sesión fraudulento o que introduzcan un código de un solo uso en un sitio web falso. Por ejemplo, un correo electrónico de *phishing* podría parecer legítimo y pedirle al usuario que introduzca un código que acaba de recibir por SMS.

- **Compromiso de la cuenta de correo o teléfono** : si un atacante tiene acceso a la cuenta de correo electrónico o al número de teléfono del usuario, puede interceptar los enlaces mágicos o los códigos de un solo uso y usarlos para iniciar sesión.

- **Ataques de *man-in-the-middle*(MITM):** un atacante podría interceptar la comunicación entre el usuario y el servidor para robar los *tokens* de autenticación o los códigos de un solo uso. Aunque menos común con las conexiones seguras, sigue siendo un riesgo.

- **Reutilización de *tokens*:** si un token o un enlace mágico puede ser usado varias veces, un atacante podría reutilizarlo para iniciar sesión repetidamente o, incluso, después de que el usuario ya haya accedido.

© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Recomendaciones de **seguridad:**

- ***Tokens* de un solo uso y con tiempo limitado**: asegurar que los enlaces mágicos y los códigos de un solo uso solo puedan ser utilizados una vez. Después del primer uso, deben ser invalidados inmediatamente.

- **Corto tiempo de vida**: establecer un tiempo de expiración muy corto para los *tokens* de autenticación (por ejemplo, 5 a 10 minutos). Esto reduce la ventana de oportunidad para un atacante que haya interceptado el *token.*

© Universidad Internacional de La Rioja (UNIR)

4.5. Autenticación single sing on

Autenticación OpenID Connect Oauth2 2.0

OAuth 2.0 es un marco de **autorización** y de **autenticación** que permite a una aplicación (el cliente) obtener **acceso limitado** a los recursos de un usuario en otro servicio (el servidor de recursos) sin compartir las credenciales del usuario. Por ejemplo, cuando una aplicación pide permiso para acceder a tu perfil de Google.

El **flujo básico** de OAuth 2.0 es el siguiente (Figura 12):

- La aplicación cliente solicita **acceso** a los **datos** de un usuario en un servicio externo.
- El usuario es **redirigido** a la página de inicio de sesión del servicio externo.
- El usuario **inicia sesión** y autoriza a la aplicación cliente a acceder a sus datos.
- El servicio externo redirige al usuario de vuelta a la aplicación cliente con un **código de autorización.**
- La aplicación cliente usa este código para solicitar un *token* de **acceso al servicio externo.**
- La aplicación cliente usa el *token* de acceso para realizar **peticiones a la API** del servicio externo en nombre del usuario.

© Universidad Internacional de La Rioja (UNIR)

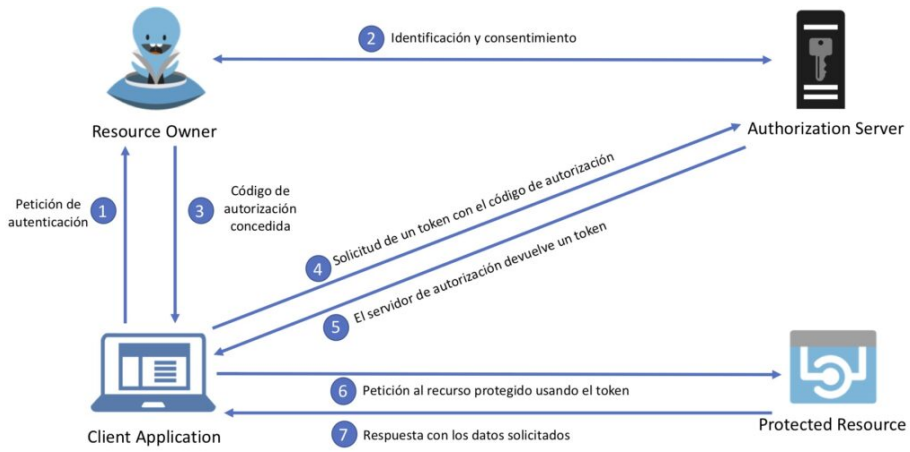


Figura 12. OpenID Connect Oauth 2.0. Fuente: Torres, 2024.

OpenID Connect (OIDC) es una **capa de autenticación** construida sobre OAuth 2.0. Proporciona una forma estándar para que las aplicaciones verifiquen la identidad de los usuarios y obtengan **información básica** del perfil. Al final del flujo, OIDC proporciona un ID *token* (un JWT) que contiene información del usuario, lo que permite a la aplicación cliente autenticar al usuario.

Ventajas: permiten el inicio de sesión único (SSO) y la autenticación a través de proveedores externos, lo que mejora la experiencia del usuario y simplifica el registro.

Desventajas: la configuración inicial puede ser compleja. Es crucial entender la diferencia entre autorización (OAuth) y autenticación (OIDC) para implementaciones correctas.

© Universidad Internacional de La Rioja (UNIR)

4.6. Autenticación. Lista de comprobación

Aspectos que tener en cuenta para implementar una **autenticación segura** (Scambray, 2011; Hoffman, 2024; Scavan, 2012):

- Identificar **datos** y **funcionalidades** que proteger (datos personales, cuentas, filiaciones, fichas médicas, transferencias, etc.).
- Identificar dentro del código dónde debería tener lugar la **autenticación.**
- Usar **contraseñas fuertes** (>12 a 16 caracteres): mayúsculas-minúsculas, números, símbolos…
- *HASH* más *SALT* (*append* al *hash* contraseña). *SALT:* pieza de información que se añade a la entrada de una función *hash* para combinarla con la contraseña y añadir dificultad de ataques contra el almacenamiento de *hashes.* Se suele almacenar como prefijo o sufijo a la contraseña. Debido a que la longitud de *SALT* es conocida, no existe problema en la verificación de la contraseña suministrada por el usuario.

Reinicio de contraseñas ante fallos de autenticación.

- Tiempo máximo de **expiración.**
- *Captcha* (ataques de fuerza bruta).
- Permitir **deshabilitar cuentas.**
- Deshabilitar cuentas por defecto.
- **Evitar recordar contraseña:** establece períodos de tiempo largos en *cookies*.
- **No incluir credenciales** en el código fuente de las aplicaciones: *hardcoded password* (Moeller, 2016; Malcom, 2024).

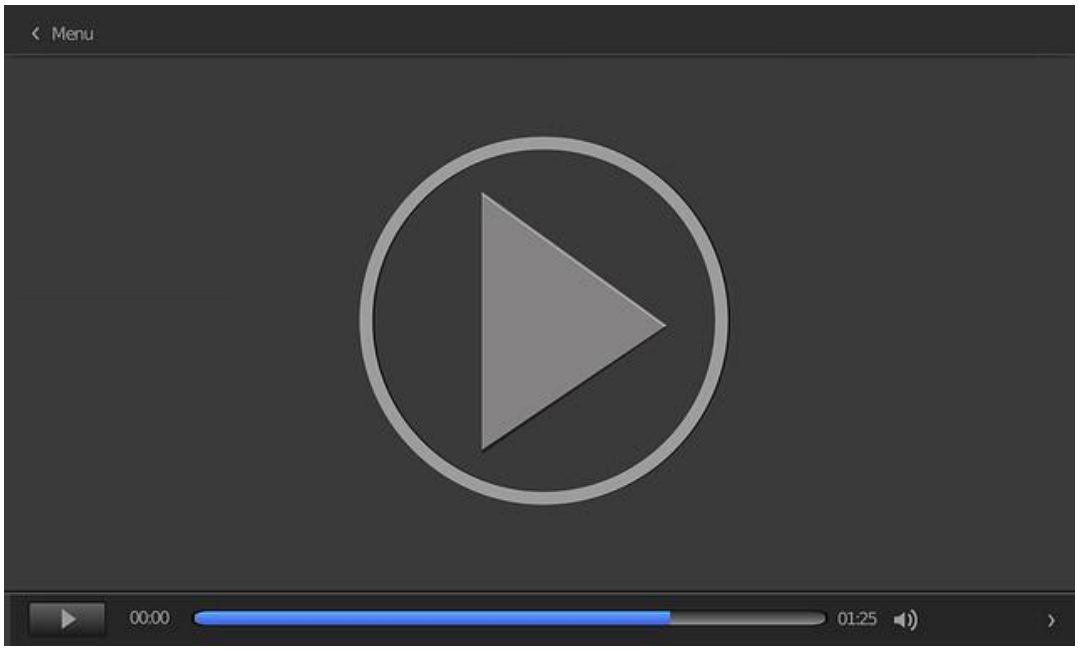
© Universidad Internacional de La Rioja (UNIR)

Ideas clave

Es importante resaltar la importancia de **combinar** varios de los **métodos de autenticación** para conseguir el mayor grado de seguridad posible, siempre adaptando el diseño a cada caso particular.

mTLS debe ser uno de los factores de autenticación **obligatorios,** teniendo en cuenta las implicaciones de rendimiento que tienen y que pueden dar lugar a denegaciones de servicio. Es conveniente aplicar TLS (HTTPS) a toda la sesión, no solo a la fase de autenticación.

Se recomienda visualizar el vídeo *OWASP top ten proactive controls* para saber más sobre los principales controles de seguridad del proyecto OWASP.



Accede al vídeo:

https://unir.cloud.panopto.eu/Panopto/Pages/Embed.aspx?id=cd6f5d9d-09ac-468d-942c-ae5d00af86bc

© Universidad Internacional de La Rioja (UNIR)

4.7. Referencias bibliográficas

Alarcón, J. M. (2015). *Autenticación de usuarios Windows y de dominio (con Directorio Activo) en IIS e IIS Express.* JASoft. https://www.jasoft.org/Blog/post/Autenticacion-de-usuarios-Windows-y-de-dominio-%28con-Directorio-Activo%29-en-IIS-e-IIS-Express

Bernat V. (2011). *TLS computational DoS mitigation*. Vincent Bernat. http://vincent.bernat.im/en/blog/2011-ssl-dos-mitigation.html#putting-more-work-on-the-client-side) (https://eprint.iacr.org/2006/212.pdf

Butler, E. (2010). *Firesheep.* Code Butler. https://codebutler.com/2010/10/24/firesheep/

Díaz Motero, C., Bermejo Higuera, J. R., Bermejo Higuera, J., Sicilia Montalvo, J. A. y Gámez Gómez, N. (2021). On attacking Kerberos authentication protocol in Windows Active Directory Services: a practical survey. *IEEE Access, 9*, 109289-109319. https://ieeexplore.ieee.org/document/9501961

Ganesh, J. (2019). *Setting up Kerberos authentication for a website in IIS.* Tech Community. https://techcommunity.microsoft.com/blog/iis-support-blog/setting-up-kerberos-authentication-for-a-website-in-iis/347882

Hoffman, A. (2024) *Web application security* (2ª ed.). O'Reilly Media.

McDonald, M. (2024). *Grokking web application security.* Simon and Schuster.

Moeller, J. P. (2016). *Security for web developers: using JavaScript, HTML and CSS.* O'Reilly Media.

© Universidad Internacional de La Rioja (UNIR)

Ochoa, H. y Azubel, A. (2010). *Understanding the Windows SMB NTLM Authentication Weak Nonce Vulnerability.* BlackHat. https://media.blackhat.com/bh-us-10/presentations/Ochoa_Azubel/BlackHat-USA-2010-Ochoa-Azubel-NTLM-Weak-Nonce-slides.pdf

Scambray, J., Liu, V., Sima, C. (2011). *Hacking exposed web applications 3.* McGraw-Hill.

Scavan, B. y Vincent, L. (2012). *Web application security, a beginner's guide*. McGraw-Hill Education.

Sobrier, J. (2010). *BlackSheep – a tool to detect Firesheep.* Zscaler. https://www.zscaler.com/es/blogs/security-research/blacksheep-tool-detect-firesheep

Solicitar un token de autenticación. (s. f.). https://processa-aspen.readthedocs.io/en/latest/JWT-Request/

Torres, G. (2024). *OAuth 2.0, OpenID Connect y JSON Web Tokens (JWT) ¿Qué es qué?* Spain Clouds. https://www.spainclouds.com/blog/oauth-2-0-openid-connect-y-json-web-tokens-jwt-que-es-que

© Universidad Internacional de La Rioja (UNIR)

Recomendaciones de seguridad para la autenticación

Authentication Cheat Sheet. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

Authenticacion Cheat Sheet es un proyecto de OWASP que da directrices y recomendaciones de seguridad para una implementación correcta de la autenticación en aplicaciones web.

© Universidad Internacional de La Rioja (UNIR)

Protocolo de autenticación Kerberos

Kerberos: the network authentication protocol. (2026). https://web.mit.edu/kerberos/

Kerberos es un protocolo de autenticación de red. Está diseñado para proporcionar una autenticación sólida para aplicaciones cliente/servidor mediante el uso de criptografía de clave secreta. El Instituto Tecnológico de Massachusetts ofrece una implementación gratuita de este protocolo. Kerberos también está disponible en muchos productos comerciales.

© Universidad Internacional de La Rioja (UNIR)

Autenticación y autorización en Azure App Service y Azure Functions

Autenticación y autorización en Azure App Service y Azure Functions. (s. f.). https://learn.microsoft.com/es-es/azure/app-service/overview-authentication-authorization

Azure App Service proporciona autenticación integrada (inicio de sesión de usuarios) y autorización (proporcionar acceso a datos seguros). Estas funcionalidades a veces se denominan Easy Auth. Puedes usarlos para iniciar sesión de usuarios y acceder a datos escribiendo poco o ningún código en la aplicación web, API RESTful, servidor móvil y funciones. En este artículo se describe cómo App Service le ayuda a simplificar la autenticación y autorización para la aplicación.

© Universidad Internacional de La Rioja (UNIR)

OWASP Transport Layer Security Cheat Sheet

OWASP Transport Layer Security Cheat Sheet. (s. f.). https://cheatsheetseries.owasp.org/cheatsheets/Transport_Layer_Security_Cheat_Sheet.html

Esta recomendación proporciona orientación sobre la implementación de la protección de la capa de transporte para aplicaciones que utilizan Transport Layer Security (TLS). Se centra en cómo utilizar TLS para proteger a los clientes que se conectan a una aplicación web a través de HTTPS, aunque gran parte de esta orientación también es aplicable a otros usos de TLS. Cuando se implementa correctamente, TLS puede proporcionar varias ventajas de seguridad:

- Confidencialidad: proporciona protección contra los atacantes que leen el contenido del tráfico.
- Integridad: proporciona protección contra la modificación del tráfico, como cuando un atacante reproduce solicitudes contra el servidor.
- Autenticación: permite al cliente confirmar que está conectado al servidor legítimo. Tenga en cuenta que la identidad del cliente no se verifica a menos que se utilicen certificados de cliente.

© Universidad Internacional de La Rioja (UNIR)

Método de autenticación OpenID Connect

What is OpenID Connect. (s. f.). https://openid.net/connect/

OpenID Connect es una simple capa de identidad en la parte superior del protocolo OAuth 2.0. Permite a los clientes verificar la identidad del usuario final basándose en la autenticación realizada por un servidor de autorización, así como obtener información básica de perfil sobre el usuario final de forma interoperable y similar a REST. OpenID Connect permite a los clientes de todo tipo, incluidos los clientes basados en la web, los móviles y los de JavaScript, solicitar y recibir información sobre las sesiones autenticadas y los usuarios finales. El conjunto de especificaciones es extensible, lo que permite a los participantes utilizar características opcionales, como la encriptación de datos de identidad, el descubrimiento de proveedores de OpenID y la gestión de sesiones, cuando tiene sentido para ellos.

© Universidad Internacional de La Rioja (UNIR)

Test

1. ¿Cómo envía *basic* las credenciales al servidor de aplicaciones?

   A. Cifrados.

   B. En claro.

   C. Codificadas en Base64.

   D. *Hash* MD5.

2. Digest usa:

   A. *Nonce.*

   B. *Response.*

   C. *Hash.*

   D. Todas las anteriores.

3. ¿Qué tipo de tiques usa Kerberos para obtener tiques de servicio?

   A. TGT.

   B. TGS.

   C. TFS.

   D. AES.

4. ¿Qué tipo de tiques usa Kerberos para acceder al servicio?

   A. TGT.

   B. TGS.

   C. TFS.

   D. AES.

© Universidad Internacional de La Rioja (UNIR)

5. ¿Cuál es el parámetro en *digest* que, bien implementado, evita ataques de repetición?

A. MD5.

B. *Cnonce*.

C. *Nonce*.

D. *Response*.

6. OAuth 2.0 usa:

A. *Token* de acceso servicio.

B. Código de autorización.

C. La A y la B.

D. Ninguna de las anteriores.

7. ¿Qué métodos de autenticación se suelen usar en Active Directory?

A. NTLM.

B. Kerberos.

C. *Digest*.

D. La A y la B.

8. Kerberos usa:

A. Base64.

B. Tiques.

C. *Tokens*.

D. *Assertion*.

© Universidad Internacional de La Rioja (UNIR)

9. ¿Cuál es un ataque a la autenticación?

A. Repetición.

B. Fuerza bruta.

C. *Man in the middle* (MITM).

D. Todas las anteriores.

10. Recomendación para una robusta autenticación:

A. Política robusta de contraseñas.

B. *Captcha*.

C. *Salt*.

D. Todas las anteriores.

© Universidad Internacional de La Rioja (UNIR)